

Gestion Sécurisée des Secrets d'Entreprise avec SecretManagement & SecretStore

1. La Dette Sécurité des Identifiants en Scripting

En administration d'infrastructure, l'automatisation exige quotidiennement de s'authentifier auprès de contrôleurs de domaine, de bases de données, d'hyperviseurs ou d'APIs Cloud (Microsoft 365, AWS, VMware).

Pendant des années, les équipes d'exploitation ont eu recours à des solutions artisanales présentant des failles critiques.

1.1 Les Trois Impasses Historiques

[1. MOT DE PASSE EN CLAIR (Clear-Text) - RISQUE CRITIQUE]

```
$Password = "P@ssw0rd2026!"
```

```
$Cred = New-Object System.Management.Automation.PSCredential("admin", ($Password | ConvertTo-SecureString -AsPlainText -Force))
```

—► Vulnérabilité : Visible par quiconque ouvre le script, tracé dans Git et exposé dans les logs d'audit 4104.

[2. CLÉ AES PARTAGÉE (Export-Clixml / ConvertFrom-SecureString avec clé)]

```
$Secure | ConvertFrom-SecureString -Key (1..16) | Out-File "C:\Sec\pass.key"
```

—► Vulnérabilité : Déplacer la clé de chiffrement dans un fichier voisin n'est pas de la sécurité (illusion de sécurité).

[3. DPAPI UTILISATEUR LOCAL (Export-Clixml sans clé)]

```
$Cred | Export-Clixml -Path "C:\Sec\cred.xml"
```

—► Limite : Chiffré avec la clé de session Windows de l'utilisateur. Dès que le script doit tourner sur un autre

serveur, sous un compte de service différent ou dans un conteneur, le déchiffrement échoue immédiatement.

1.2 Les Exigences d'une Gestion de Secrets d'Entreprise

Une gestion industrielle des secrets doit satisfaire quatre critères fondamentaux :

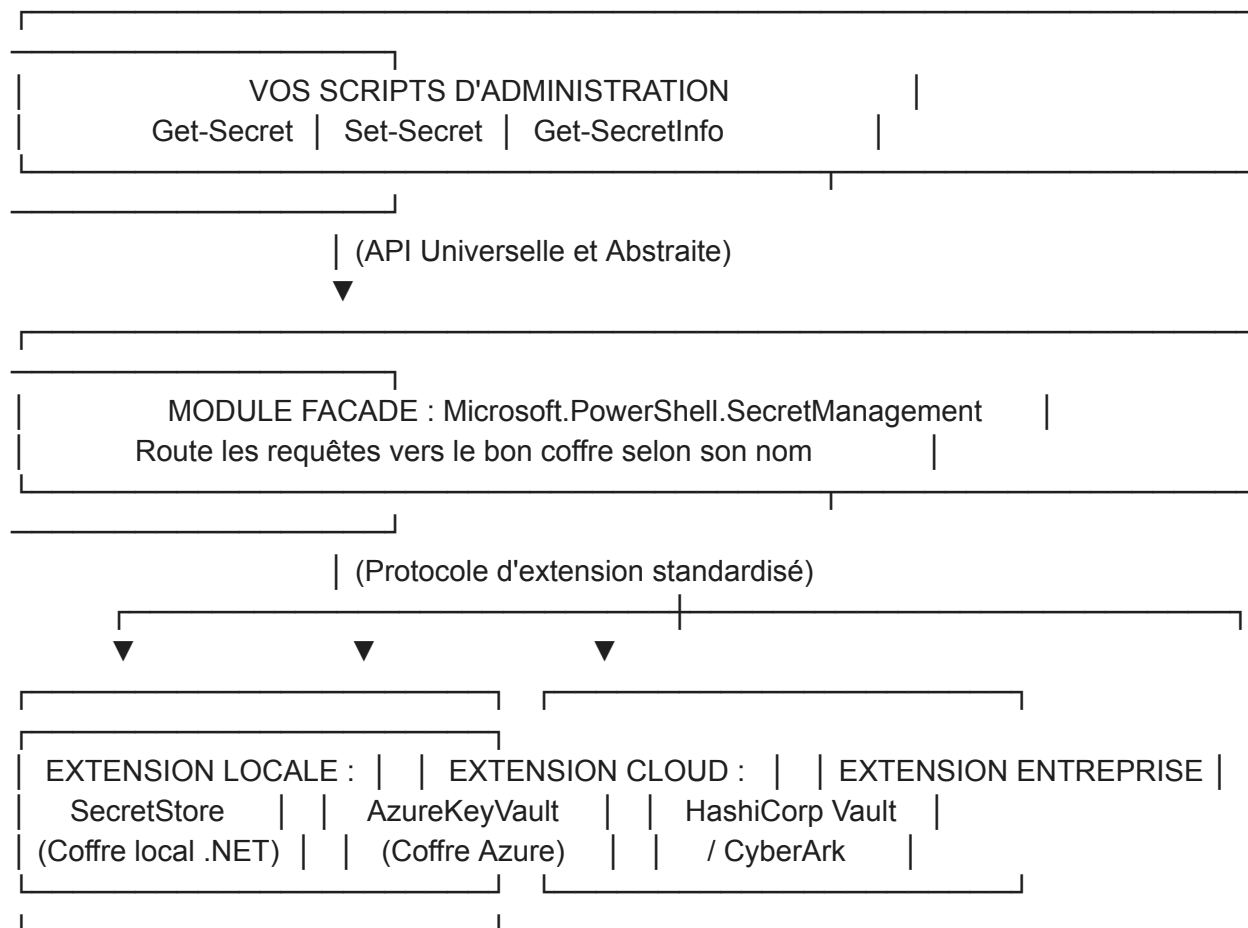
1. **Zéro secret dans le code source** : Aucun mot de passe ou clé privée ne doit figurer dans les scripts .ps1 ni dans les dépôts Git.
2. **Chiffrement fort au repos** : Les données sensibles doivent être stockées dans un

magasin protégé par un algorithme cryptographique robuste (ex. AES-256).

3. **Indépendance du fournisseur (*Vendor Agnostic*)** : Le code d'automatisation doit être identique, que le secret soit extrait d'un coffre local sur un poste de test ou d'un coffre d'entreprise distant (Azure Key Vault, HashiCorp Vault, CyberArk).
4. **Support des tâches sans assistance (*Unattended Scripts*)** : Capacité à déverrouiller le coffre sans intervention humaine pour les tâches planifiées de nuit et les pipelines CI/CD.

2. L'Architecture Découplée de Microsoft

Pour répondre à ces enjeux, Microsoft a introduit une architecture modulaire scindée en **deux couches distinctes** :



2.1 Le Rôle de Microsoft.PowerShell.SecretManagement

C'est le module d'**abstraction universel** (la façade). Il expose les cmdlets standards que vous utilisez dans vos scripts (Get-Secret, Set-Secret, Register-SecretVault).

- Il ne stocke **aucun** secret par lui-même.

- Il agit comme un routeur qui transmet l'ordre au coffre physique sous-jacent.

2.2 Le Rôle de Microsoft.PowerShell.SecretStore

C'est le **fournisseur de coffre local** par défaut conçu par Microsoft.

- Il implémente physiquement le magasin sécurisé sur la machine locale.
- Il stocke les secrets chiffrés dans le profil de l'utilisateur sous forme de fichier chiffré (%LOCALAPPDATA%\Microsoft\PowerShell\secretstore).
- Il est multiplateforme (Windows, Linux, macOS).

Bénéfice architectural majeur : Vous écrivez votre script avec `Get-Secret -Name "CompteSauvegarde"`. En environnement de développement, la commande interroge votre coffre local SecretStore. En production, le même script interroge un coffre Azure Key Vault sans modifier une seule ligne de code PowerShell.

3. Installation et Enregistrement du Coffre

3.1 Installation des Deux Modules Officiels

Ouvrez une invite PowerShell et installez les modules depuis PowerShell Gallery :

```
# Installation de la façade et du fournisseur local
Install-Module -Name Microsoft.PowerShell.SecretManagement -Repository PSGallery -Scope
CurrentUser -Force
Install-Module -Name Microsoft.PowerShell.SecretStore -Repository PSGallery -Scope
CurrentUser -Force
```

Vérifiez la disponibilité des modules en mémoire :

```
Get-Module -ListAvailable Microsoft.PowerShell.Secret*
```

3.2 Enregistrement du Coffre Local (Register-SecretVault)

Avant de pouvoir enregistrer des secrets, vous devez déclarer votre coffre local auprès du gestionnaire SecretManagement :

```
# Enregistrement du coffre local comme coffre par défaut
Register-SecretVault -Name "LocalSecVault" `
    -ModuleName "Microsoft.PowerShell.SecretStore" `
    -DefaultVault
```

Paramètres clés de Register-SecretVault :

- **-Name** : Nom logique et convivial attribué au coffre (ex. LocalSecVault, ProdAzureVault).
- **-ModuleName** : Nom du module d'extension qui gère physiquement le stockage (Microsoft.PowerShell.SecretStore).
- **-DefaultVault** : Marque ce coffre comme destination implicite. Toutes les commandes Get-Secret ou Set-Secret qui n'indiquent pas de paramètre -Vault s'adresseront automatiquement à lui.

Pour lister les coffres actuellement enregistrés :

```
Get-SecretVault
```

4. Configuration et Durcissement du Coffre (Set-SecretStoreConfiguration)

Le module SecretStore offre une granularité de sécurité configurable via la cmdlet **Set-SecretStoreConfiguration**. La configuration détermine comment le coffre est verrouillé, déverrouillé et surveillé en mémoire.

4.1 Afficher la Configuration Active

```
Get-SecretStoreConfiguration
```

Propriétés fondamentales :

Paramètre	Rôle technique	Valeurs possibles
Scope	Portée du coffre (espace utilisateur)	CurrentUser
Authentication	Méthode requise pour ouvrir le coffre	Password ou None
PasswordTimeout	Délai en secondes avant reverrouillage automatique en RAM	Entier (ex: 900 pour 15 min, -1 pour infini)
PromptForPassword	Invite interactive à saisir le mot de passe si verrouillé	\$true ou \$false
ConfirmPrompt	Demande confirmation pour chaque écriture de secret	\$true ou \$false

4.2 Les Deux Modes Opérationnels : Interactif vs Automatisé

Mode A : Le Mode Interactif Durci (Poste de Travail / Développeur)

Ce mode exige un mot de passe maître pour ouvrir le coffre. Une fois saisi, le coffre reste ouvert en mémoire vive pendant la durée fixée par PasswordTimeout (par défaut 900 secondes = 15 minutes) :

```
Set-SecretStoreConfiguration -Authentication Password `
    -PasswordTimeout 900 `
    -PromptForPassword:$true `
    -ConfirmPrompt:$false
```

- Lors du premier accès, PowerShell vous invite à définir un mot de passe maître fort.
- Si le délai expire, toute commande Get-Secret redemandera le mot de passe maître via une invite sécurisée.

Mode B : Le Mode Non-Surveillé (*Unattended* / Tâches Planifiées / CI/CD)

Sur un serveur de production où un script s'exécute la nuit sans opérateur humain pour taper un mot de passe, l'authentification par mot de passe maître est inapplicable.

On configure le coffre avec **Authentication None** :

```
Set-SecretStoreConfiguration -Authentication None `
    -Interaction None `
    -ConfirmPrompt:$false
```

- **Comment le secret est-il protégé sans mot de passe ?**
Sous Windows, SecretStore utilise alors l'API cryptographique **DPAPI** liée au compte de service qui exécute la session. Le fichier physique sur disque reste chiffré, mais le déchiffrement s'effectue automatiquement et exclusivement pour ce compte utilisateur spécifique sur cette machine précise.

5. Le Cycle de Vie des Secrets : Manipulation Pratique

Le gestionnaire de secrets prend en charge plusieurs types d'objets sensibles natifs :

- [PSCredential] : Objet couple identifiant / mot de passe.
- [SecureString] : Chaîne chiffrée en mémoire.
- [string] : Jeton d'API, token JWT, mot de passe brut ou URL secrète.
- [hashtable] : Dictionnaire de paramètres de configuration chiffrés.
- [byte[]] : Clés binaires privées ou certificats.

5.1 Créer ou Mettre à Jour un Secret (Set-Secret)

Exemple 1 : Enregistrer une clé d'API (String)

```
Set-Secret -Name "MonitoringApiKey" -Secret "a8f7c9e1204d49a78bc56123ef"
```

Exemple 2 : Enregistrer un compte de service ([PSCredential])

```
$Credential = Get-Credential -UserName "svc_backup" -Message "Entrez le mot de passe du  
compte de sauvegarde"
```

```
Set-Secret -Name "CompteSauvegarde" -Secret $Credential
```

Exemple 3 : Associer des métadonnées de conformité

On peut enrichir le secret avec des informations de contexte sans exposer le mot de passe :

```
$Metadonnees = @{  
    Application = "SauvegardeVeeam"  
    Responsable = "Equipe_Infra"  
    ExpireLe    = "2027-12-31"  
}
```

```
Set-Secret -Name "CompteSauvegarde" -Secret $Credential -Metadata $Metadonnees
```

5.2 Explorer le Catalogue sans Révéler les Secrets (Get-SecretInfo)

En audit ou en supervision, on doit pouvoir inspecter les secrets présents sans charger leurs valeurs en mémoire vive. C'est le rôle de **Get-SecretInfo** :

```
Get-SecretInfo
```

Sortie console type :

Name	Type	VaultName	Metadata
CompteSauvegarde	PSCredential	LocalSecVault	{[Application, SauvegardeVeeam], ...}
MonitoringApiKey	String	LocalSecVault	{}

5.3 Extraire et Consommer un Secret (Get-Secret)

Extraction d'un compte de service :

```
$MonCompte = Get-Secret -Name "CompteSauvegarde"
```

```
# $MonCompte est un véritable objet [System.Management.Automation.PSCredential]
```

```
$MonCompte.UserName
```

```
# Le mot de passe reste protégé en mémoire vive dans un objet SecureString :
```

```
$MonCompte.Password
```

Extraction sous forme de texte brut (-AsPlainText) :

Certaines commandes ou requêtes web (ex. en-têtes d'authentification HTTP Authorization: Bearer <Token>) attendent une chaîne de texte simple. Le paramètre -AsPlainText évite d'avoir à manipuler des pointeurs mémoire complexes :

```
$TokenAPI = Get-Secret -Name "MonitoringApiKey" -AsPlainText
```

Attention sécurité : Limitez au strict minimum la durée de vie des variables contenant du texte brut déchiffré. Écrivez-les dès que l'appel technique est terminé :

```
$TokenAPI = $null
```

5.4 Déverrouillage Manuel et Gestion de Session

Si votre coffre est configuré avec un mot de passe maître, vous pouvez le déverrouiller programmatiquement en début de session :

```
# Déverrouiller explicitement le magasin pour la durée de session
```

```
Unlock-SecretStore
```

```
# Réinitialiser / Changer le mot de passe maître du coffre
```

```
Set-SecretStorePassword
```

5.5 Supprimer un Secret et Nettoyer

```
# Supprimer un secret précis
```

```
Remove-Secret -Name "MonitoringApiKey"
```

```
# Révoquer l'accès à un coffre complet
```

```
Unregister-SecretVault -Name "LocalSecVault"
```

6. Cas Pratiques de Production

Cas Pratique 1 : Connexion API REST avec Jeton Chiffré

Voici un script d'audit qui interroge une API de supervision d'infrastructure sans jamais divulguer le jeton dans le code :

```
[CmdletBinding()]
param(
    [string]$UriSupervision = "https://api.infra.entreprise.local/v1/sante"
)

process {
    Write-Verbose "Récupération sécurisée du jeton API depuis le coffre local..."

    try {
        # Extraction du secret sous forme de chaîne protégée
        $CleAPI = Get-Secret -Name "MonitoringApiKey" -AsPlainText -ErrorAction Stop
    }
    catch {
        Write-Error "Impossible d'extraire la clé API du coffre : $($_.Exception.Message)"
        return
    }

    # Préparation des en-têtes sécurisés
    $Headers = @{
        "Authorization" = "Bearer $CleAPI"
        "Accept"       = "application/json"
    }

    try {
        Write-Verbose "Interrogation de l'API : $UriSupervision"
        $Reponse = Invoke-RestMethod -Uri $UriSupervision -Headers $Headers -Method Get
    }
    catch {
        Write-Error "Échec de l'appel REST : $($_.Exception.Message)"
    }
    finally {
        # Purge immédiate de la variable mémoire sensible
    }
}
```

```
$CleAPI = $null
$headers = $null
}
}
```

Cas Pratique 2 : Tâche Planifiée Windows Non-Surveillée (*Unattended Scheduled Task*)

Pour faire tourner un script sous un compte de service dédié (ex. CONTOSO\svc-scriptrunner) sans invite interactive :

Étape 1 : Initialisation unique sous la session du compte de service

Ouvrez une session avec le compte de service svc-scriptrunner et configurez le coffre en mode silencieux :

```
# Exécuté une seule fois sous l'identité du compte de service
Install-Module Microsoft.PowerShell.SecretManagement -Scope CurrentUser -Force
Install-Module Microsoft.PowerShell.SecretStore -Scope CurrentUser -Force

Register-SecretVault -Name "BatchVault" -ModuleName "Microsoft.PowerShell.SecretStore"
-DefaultVault

# Neutralisation de toute invite interactive
Set-SecretStoreConfiguration -Authentication None -Interaction None -ConfirmPrompt:$false

# Enregistrement du compte Active Directory utilisé pour administrer le domaine
$AdminCred = Get-Credential -UserName "CONTOSO\adm_backup"
Set-Secret -Name "ADSyncCredentials" -Secret $AdminCred
```

Étape 2 : Le script exécuté par le Planificateur de Tâches

Le script de production s'exécute la nuit de manière autonome et sans blocage :

```
# Script de nuit : run_backup.ps1
$Creds = Get-Secret -Name "ADSyncCredentials"

# Utilisation directe dans une commande d'infrastructure
Invoke-Command -ComputerName "SRV-DC01" -Credential $Creds -ScriptBlock {
    Get-Service -Name "NTDS"
}
```

7. Tableau Comparatif des Stratégies de Stockage

Critère d'évaluation	Fichier texte / Code brut	Export-Clixml (DPAPI brut)	SecretStore (Local)	Azure Key Vault (Cloud)
Sécurité cryptographique	Nulle (0/5)	Moyenne (3/5)	Élevée (4.5/5)	Maximale (5/5)
Portabilité du code	Nulle	Faible (lié à la machine)	Universelle (même cmdlet)	Universelle (même cmdlet)
Gestion des métadonnées	Aucune	Aucune	Oui (table de hachage)	Oui (Tags Azure)
Indépendance de l'OS	Oui	Non (Windows uniquement)	Oui (Multiplateforme)	Oui (Multiplateforme)
Audit des accès	Aucun	Aucun	Logs locaux	Audit logs complets (SIEM)
Cas d'usage cible	À proscrire formellement	Développements temporaires	Serveurs locaux, labs, CI/CD	Entreprise, Cloud, Hybride

8. Synthèse des Bonnes Pratiques de Sécurité

1. **Adopter systématiquement la façade SecretManagement** : N'appellez jamais les fonctions internes de SecretStore directement. Utilisez exclusivement Get-Secret et Set-Secret pour conserver un code portable.
2. **Bannir les mots de passe maîtres faibles en mode interactif** : Si vous utilisez Authentication Password, choisissez une phrase de passe d'au moins 16 caractères.
3. **Purger les variables en mémoire** : Si vous devez utiliser -AsPlainText, réinitialisez la variable à \$null immédiatement après l'exécution de la commande cible dans un bloc finally.
4. **Isoler les coffres par compte de service** : Deux comptes Windows distincts sur la même machine possèdent des coffres étanches. Ne tentez pas de partager un fichier physique de coffre entre plusieurs identités.
5. **Auditer les secrets avec Get-SecretInfo** : Planifiez un contrôle régulier pour inventorier les secrets obsolètes sans avoir besoin d'afficher leurs mots de passe.